

Assignment 4.2 Ai Assisted Coding

Htno:2303a510j7

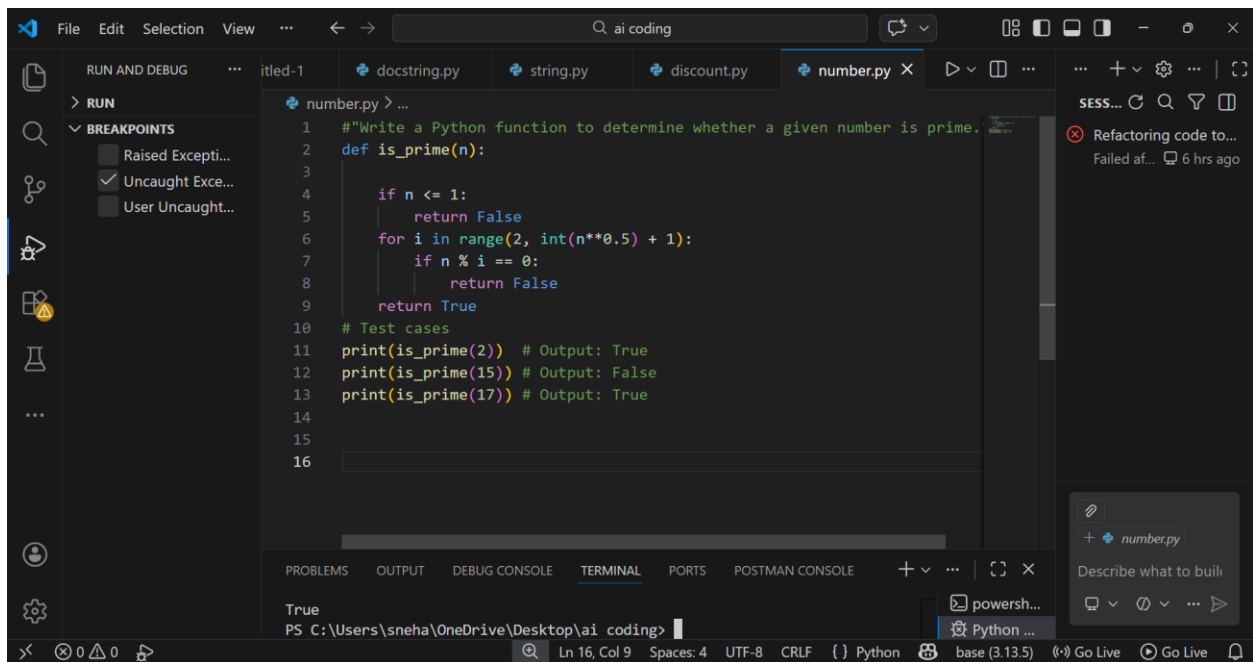
Btno:30

Task 1:

Prompt:

Write a Python function to determine whether a given number is prime.

Code:

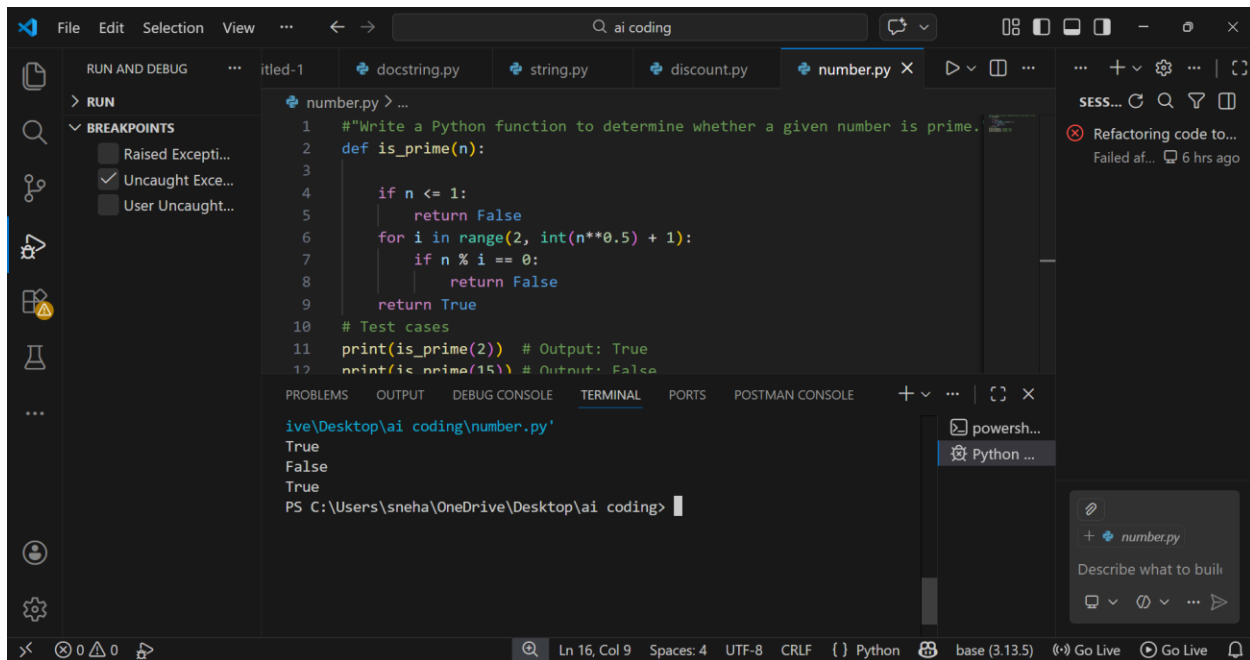
A screenshot of the Visual Studio Code (VS Code) editor interface. The main editor window displays a Python file named 'number.py'. The code defines a function 'is_prime(n)' that checks if a number is prime. It includes test cases for 2, 15, and 17. The output of the test cases is shown in the terminal at the bottom. The interface includes a sidebar with 'RUN AND DEBUG' and 'BREAKPOINTS' sections, a top bar with search and view icons, and a bottom status bar showing file encoding and language settings.

```
1  """Write a Python function to determine whether a given number is prime."""
2  def is_prime(n):
3
4      if n <= 1:
5          return False
6      for i in range(2, int(n**0.5) + 1):
7          if n % i == 0:
8              return False
9      return True
10
11 # Test cases
12 print(is_prime(2)) # Output: True
13 print(is_prime(15)) # Output: False
14 print(is_prime(17)) # Output: True
15
16
```

True

PS C:\Users\sneha\OneDrive\Desktop\ai coding>

Output:



Observation:

-->AI produced correct logic even without examples, but structure depends fully on instruction clarity.

-->Output correctness is good, but formatting and approach can change depending on how the question is worded.

Task 2:

Prompt:

Example: Input: [1,2,3,4], Output: 10. Write a Python function that calculates the sum of elements in a list.

Code:

```
File Edit Selection View ... ai coding
EXPLORER
  AI CODING
    .cursor
    .vscode
    discount.py
    docstring.py
    even number.py
    evennumber.py
    fibonacci.py
    file_operations.py
    number.py
    prime_docstring.py
    sample.txt
    string.py
    sum.py
  sum.py
    1 #Example: Input: [1,2,3,4], Output: 10. Write a Python function that ca
    2 def calculate_sum(numbers):
    3     if not all(isinstance(num, (int, float)) for num in numbers):
    4         return "Invalid input. All elements must be numbers."
    5     else:
    6         return sum(numbers)
    7 # Test cases
    8 print(calculate_sum([1, 2, 3, 4])) # Output: 10
    9
    10 print(calculate_sum([1, '2', 3, 4])) # Output: Invalid input. All elem
    11 print(calculate_sum([1.5, 2.5, 3.0])) # Output: 7.0
    12
    13
    14
  PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
  debugpy\launcher '60512' '--' 'C:\Users\sneha\OneDrive\Desktop\ai coding\s
  um.py'
```

Output:

```
File Edit Selection View ... ai coding
EXPLORER
  AI CODING
    .cursor
    .vscode
    discount.py
    docstring.py
    even number.py
    evennumber.py
    fibonacci.py
    file_operations.py
    number.py
    prime_docstring.py
    sample.txt
    string.py
    sum.py
  sum.py
    1 #Example: Input: [1,2,3,4], Output: 10. Write a Python function that ca
    2 def calculate_sum(numbers):
    3     if not all(isinstance(num, (int, float)) for num in numbers):
    4         return "Invalid input. All elements must be numbers."
    5     else:
    6         return sum(numbers)
    7 # Test cases
    8 print(calculate_sum([1, 2, 3, 4])) # Output: 10
    9
    10 print(calculate_sum([1, '2', 3, 4])) # Output: Invalid input. All elem
    11 print(calculate_sum([1.5, 2.5, 3.0])) # Output: 7.0
    12
    13
    14
  PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
  PS C:\Users\sneha\OneDrive\Desktop\ai coding>
  PS C:\Users\sneha\OneDrive\Desktop\ai coding> python sum.py
  10
  Invalid input. All elements must be numbers.
  7.0
  PS C:\Users\sneha\OneDrive\Desktop\ai coding>
```

Observation:

-->One-shot prompting helped AI clearly understand the required operation.

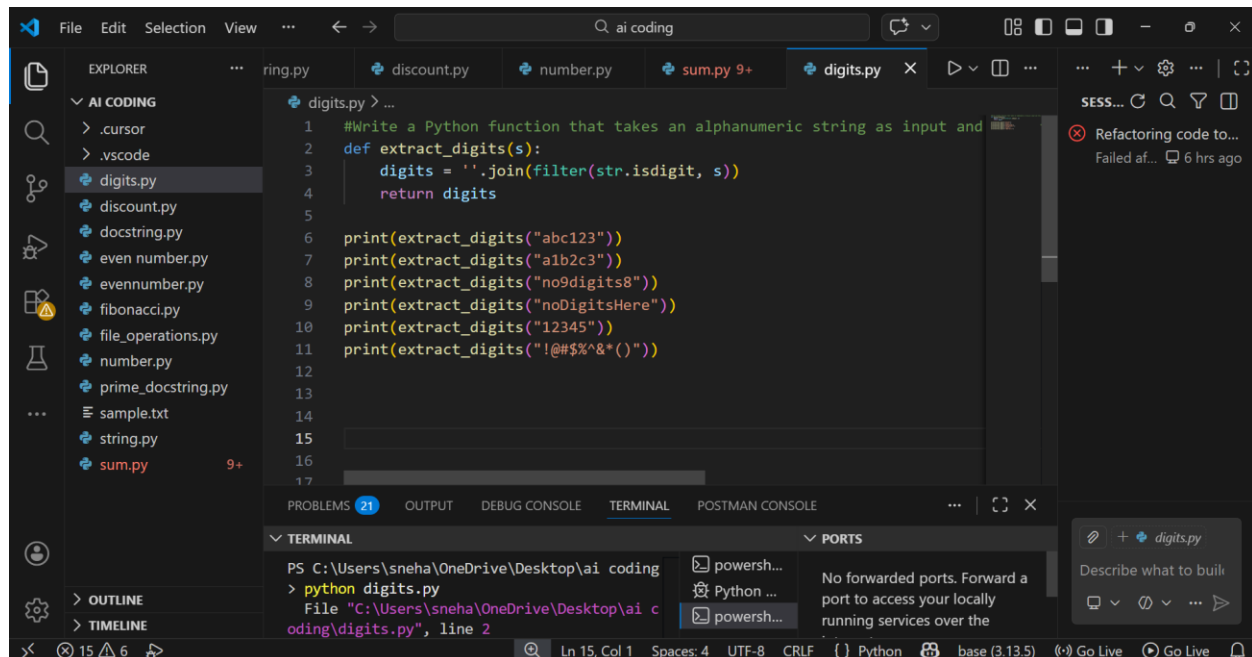
--> AI generalized the rule from the single sample.

Task 3:

Prompt

Write a Python function that takes an alphanumeric string as input and returns a string containing only the digits from it.

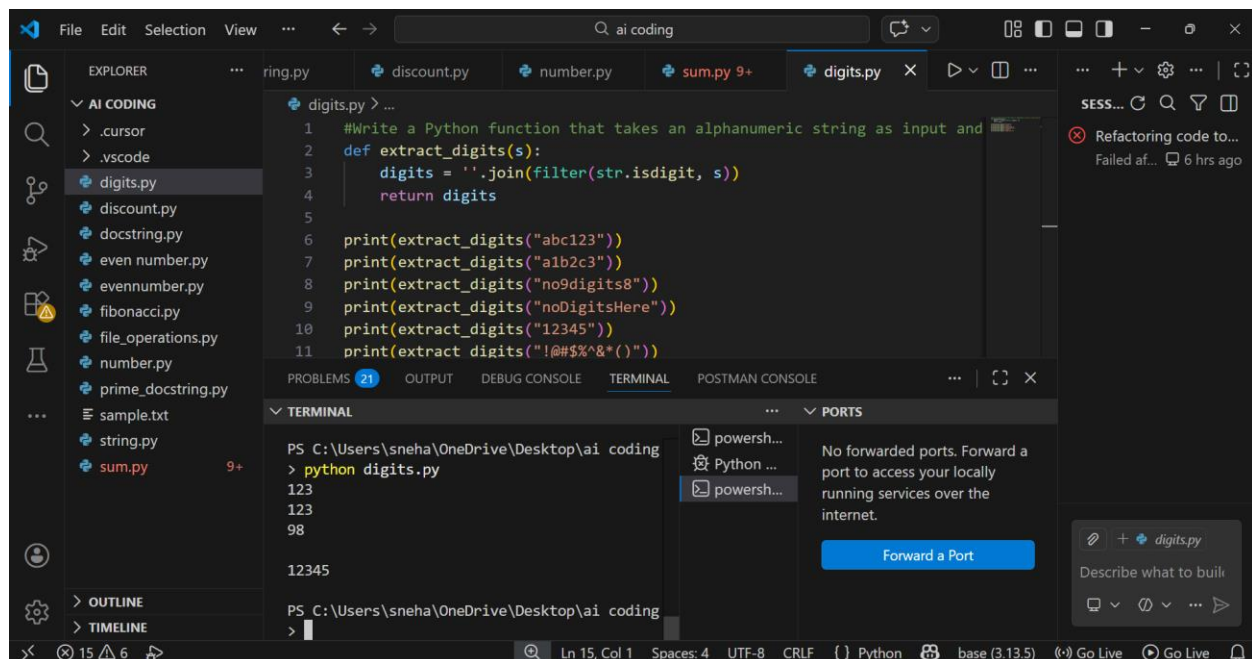
Code:



The screenshot shows the Visual Studio Code editor with a file named `digits.py` open. The code defines a function `extract_digits(s)` that uses `filter(str.isdigit, s)` to extract digits from a string `s` and joins them back into a single string. The function is tested with several examples: `abc123`, `a1b2c3`, `no9digits8`, `noDigitsHere`, `12345`, and a string with various special characters. The terminal output shows the results of these tests.

```
1 #Write a Python function that takes an alphanumeric string as input and
2 def extract_digits(s):
3     digits = ''.join(filter(str.isdigit, s))
4     return digits
5
6 print(extract_digits("abc123"))
7 print(extract_digits("a1b2c3"))
8 print(extract_digits("no9digits8"))
9 print(extract_digits("noDigitsHere"))
10 print(extract_digits("12345"))
11 print(extract_digits("!@#%$^&*()"))
```

Output:



The screenshot shows the same Visual Studio Code editor with the `digits.py` file. The terminal output now shows the results of running the code. The output is as follows:

```
PS C:\Users\sneha\OneDrive\Desktop\ai coding
> python digits.py
123
123
98
12345
PS C:\Users\sneha\OneDrive\Desktop\ai coding
>
```

Observation:

--> Few-shot examples improved accuracy and clarity of logic.

--> Multiple examples showed pattern: **keep only digits.**

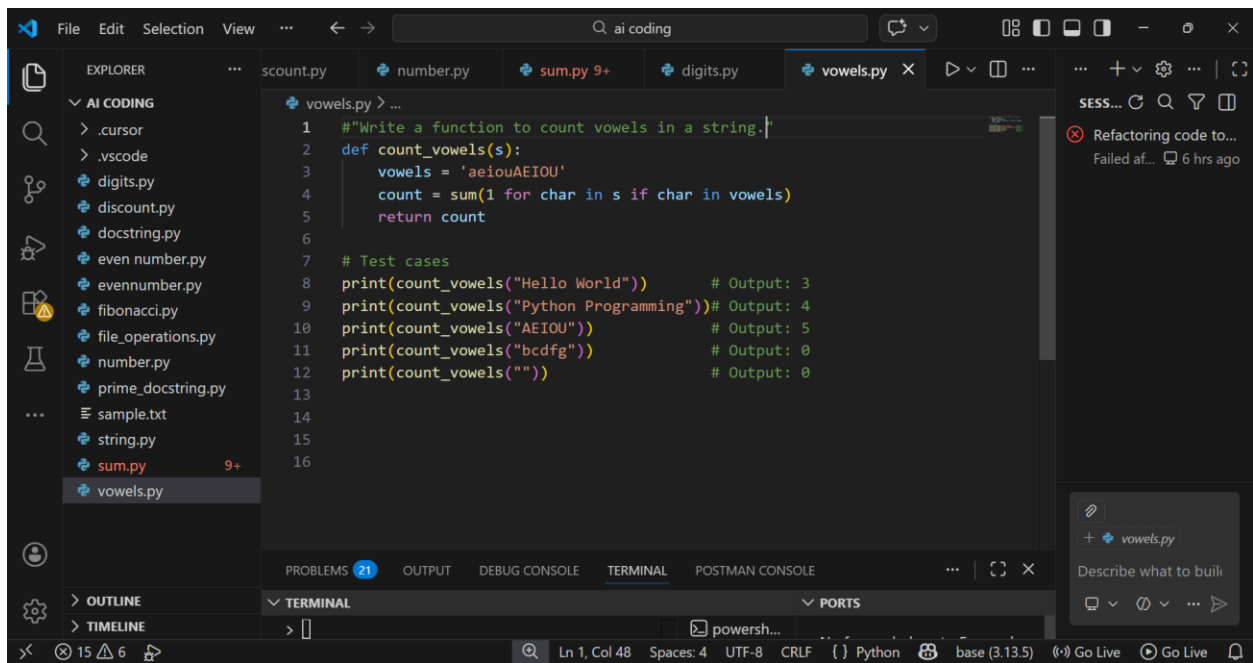
--> AI used `.isdigit()` for filtering.

Task 4:

Prompt: one short prompt

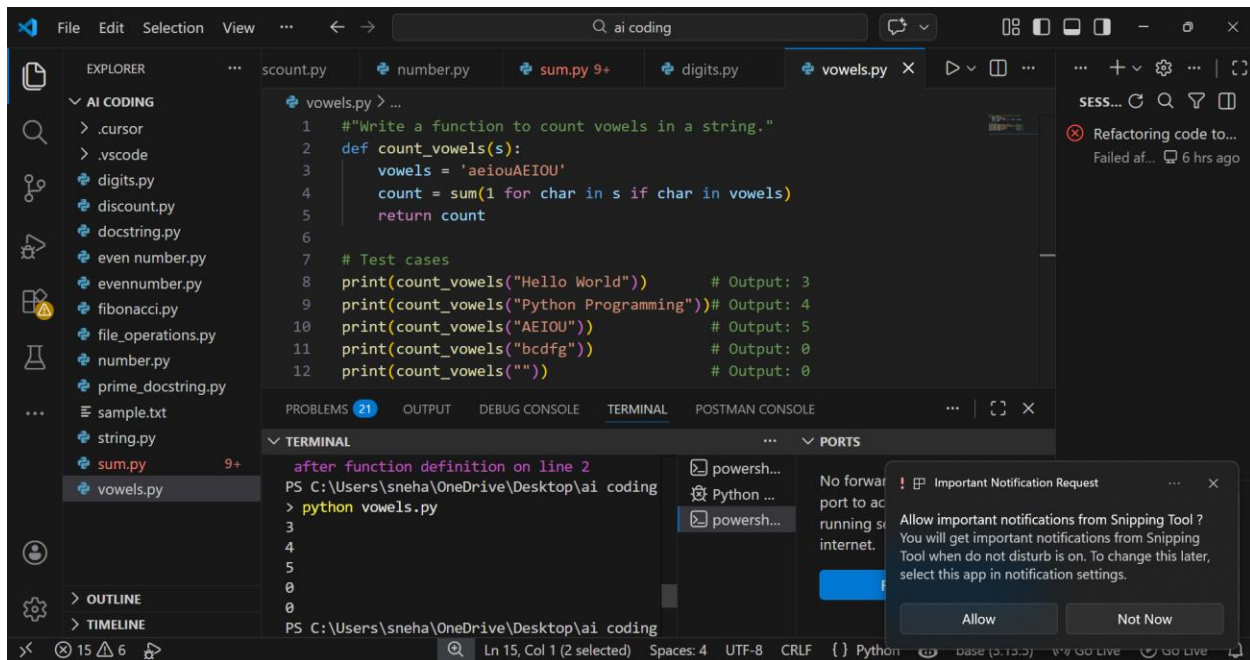
a) Write a function to count vowels in a string.

Code:

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'AI CODING' with various files, including 'vowels.py' which is currently selected. The main editor window displays the code for 'vowels.py'. The code defines a function 'count_vowels(s)' that counts the number of vowels in a string 's'. It uses a set of vowels 'aeiouAEIOU' and a sum of boolean checks. Below the function definition, there are test cases using 'print' statements to verify the function's output for different inputs. The bottom status bar shows the current cursor position at line 1, column 48, and the file is encoded in UTF-8 with CRLF line endings. A search bar at the top contains the text 'ai coding'.

```
1  #Write a function to count vowels in a string.
2  def count_vowels(s):
3      vowels = 'aeiouAEIOU'
4      count = sum(1 for char in s if char in vowels)
5      return count
6
7  # Test cases
8  print(count_vowels("Hello World"))      # Output: 3
9  print(count_vowels("Python Programming"))# Output: 4
10 print(count_vowels("AEIOU"))            # Output: 5
11 print(count_vowels("bcd fg"))           # Output: 0
12 print(count_vowels(""))                 # Output: 0
13
14
15
16
```

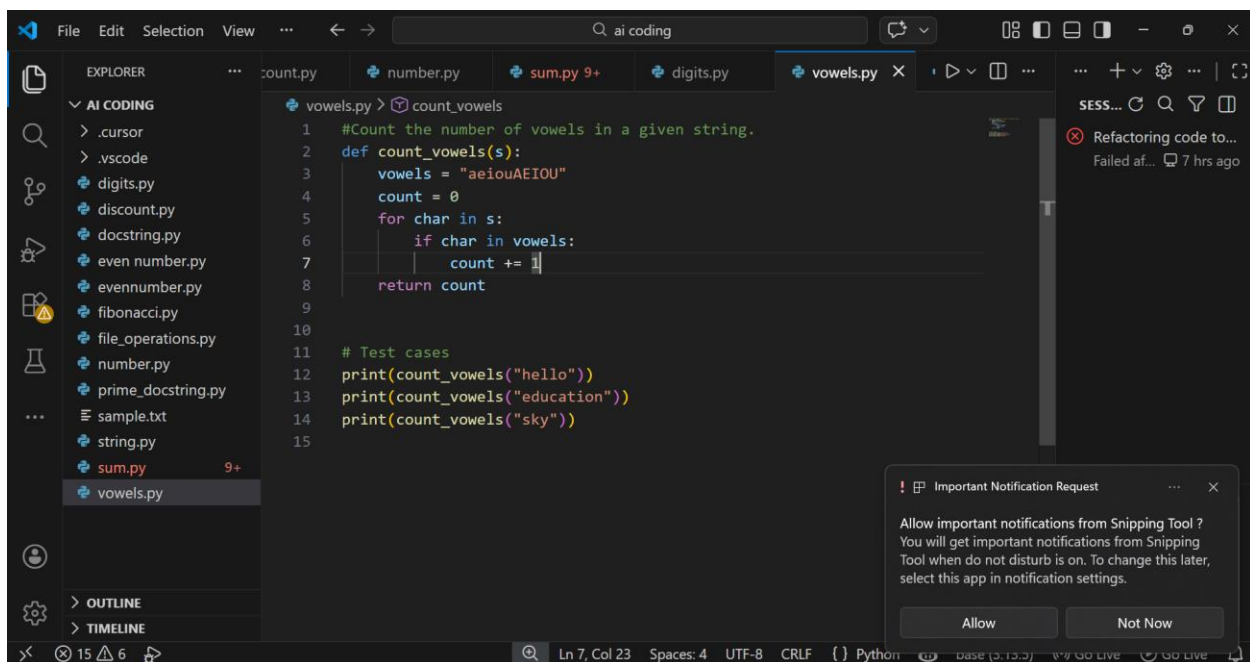
Output:



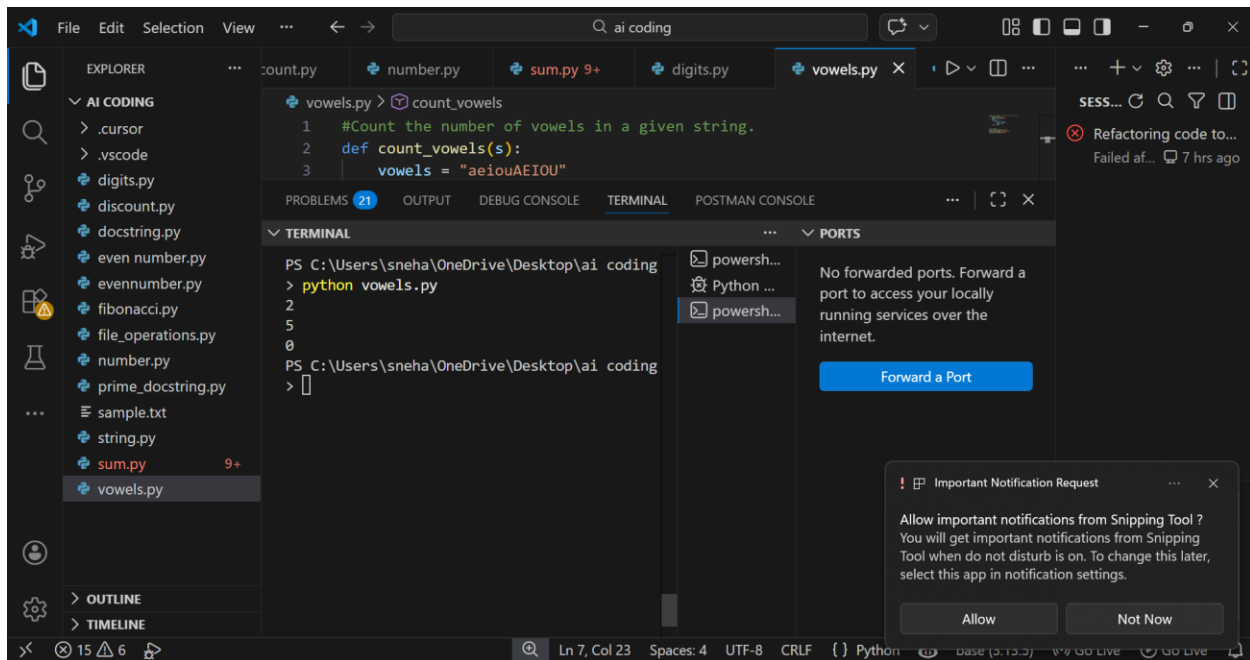
B)few short prompt:

Count the number of vowels in a given string.

Code:



Output:



Observation:

-->**Zero-shot:** AI generated correct logic, but structure depended only on the clarity of the instruction.

--> **Few-shot:** AI produced more accurate and structured output because examples guided the pattern clearly.

-->**Zero-shot:** May miss edge cases if instruction is not detailed.

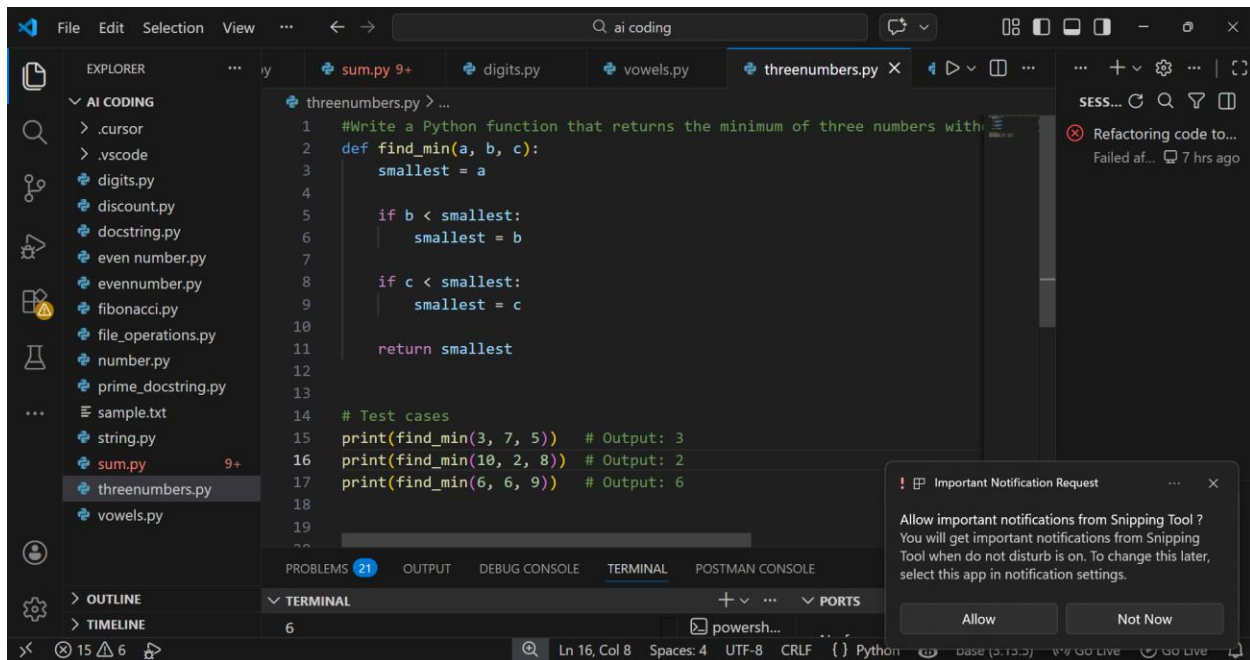
--> **Few-shot:** Better handling of different cases due to example-based learning.

Task 5:

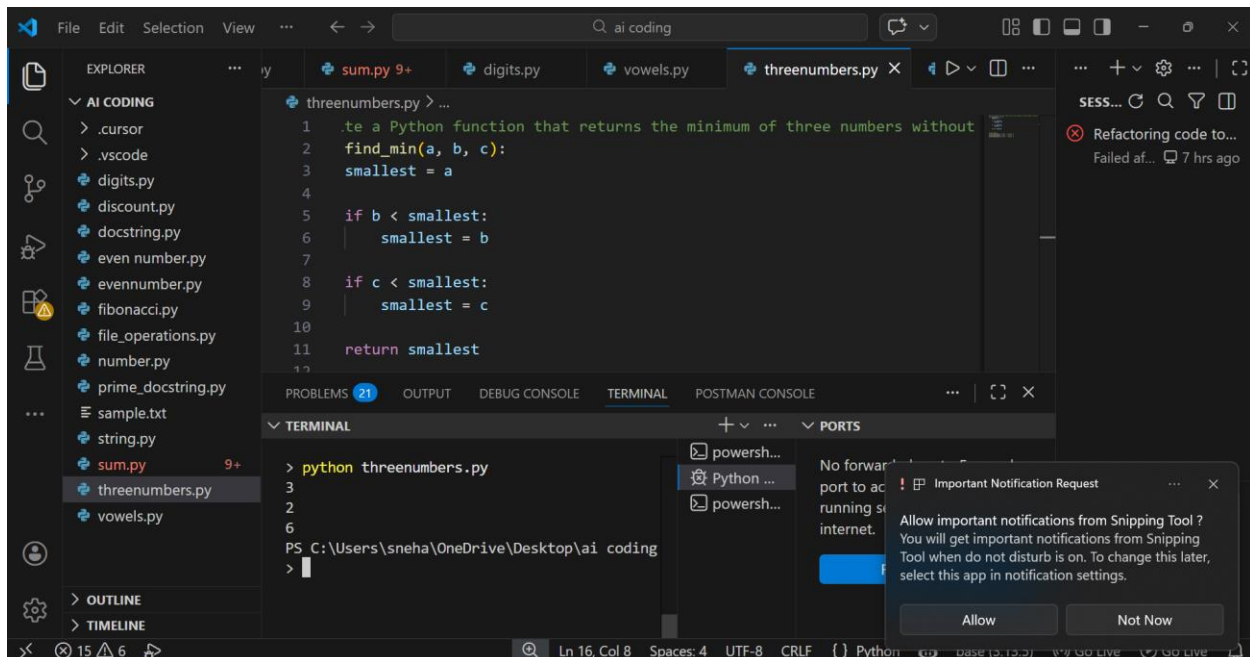
Prompt:

Write a Python function that returns the minimum of three numbers without using the built-in `min()` function.

Code:



Output:



Observation:

--> The function correctly finds the smallest value using comparison logic instead of the built-in `min()` function.

--> It works for different number combinations, including when two numbers are equal.

--> The step-by-step comparison approach makes the logic simple and easy to understand.
